

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально-науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 2
з навчальної дисципліни
"ТЕХНОЛОГІЇ РОЗРОБКИ КОРПОРАТИВНИХ
WEB-ДОДАТКІВ"

Виконала:
Студентка 402-ТН
Петрик Т.Є.
Перевірив:
Тютюнник П.Б.

Полтава - 2026

Завдання

1. Обґрунтування вибору технологічного стеку (Technology Stack) На основі вимог, зібраних у першій лабораторній роботі, оберіть технології для реалізації проекту.

- **Створіть документ або розділ у звіті, де чітко визначте:**
 - **Мова програмування Back-end:** (наприклад, C#/.NET, Java/Spring, Node.js, Python/Django).
 - **Фреймворк Front-end:** (наприклад, React, Angular, Vue.js або Server-Side Rendering — Razor/Thymeleaf).
 - **Система керування базами даних (СКБД):** (PostgreSQL, MS SQL, MongoDB, Redis тощо).
 - **Інструменти DevOps/Hosting:** (Docker, GitHub Actions, Azure/AWS — опціонально).
- **Надайте обґрунтування:** Чому саме ці технології підходять для вирішення вашої бізнес-задачі? (Наприклад: "Обрано MongoDB, оскільки структура даних каталогу товарів є гнучкою та часто змінюється" або "Обрано .NET через надійну типізацію та швидкість розробки корпоративних API").

2. Архітектурне моделювання за методологією C4 (C4 Model) Спроектуйте архітектуру вашої системи, створивши три рівні діаграм C4. Це допоможе зрозуміти структуру системи від загального до конкретного.

- **Рівень 1: System Context Diagram (Контекст).**
 - Покажіть вашу систему як одну "чорну скриньку" в центрі.
 - Відобразіть усіх користувачів (Actors) та зовнішні системи (наприклад, платіжні шлюзи, API поштових розсилок), з якими вона взаємодіє.
- **Рівень 2: Container Diagram (Контейнери).**
 - "Відкрийте" систему і покажіть її основні технічні блоки (контейнери).
 - Це можуть бути: Single Page Application, Mobile App, API Application, Database, File System.

- Вкажіть технології для кожного контейнера та протоколи взаємодії між ними (HTTP/HTTPS, TCP, GRPC).
- **Рівень 3: Component Diagram (Компоненти).**
 - Деталізуйте **один** основний контейнер (наприклад, API Application).
 - Покажіть внутрішні компоненти (наприклад: AuthController, EmailService, UserRepository) та їхні зв'язки.

3. Проєктування бази даних (ER-Model / Data Schema) Спроєктуйте схему зберігання даних. Вибір типу діаграми залежить від обраної СКБД:

- **Варіант А: Реляційна база даних (SQL)**
 - Побудуйте класичну **ER-діаграму (Entity-Relationship Diagram)**.
 - Вкажіть сутності, атрибути, первинні (PK) та зовнішні (FK) ключі.
 - Визначте типи зв'язків (1:1, 1:N, M:N).
 - Забезпечте нормалізацію даних (мінімум до 3-ї нормальної форми).
- **Варіант Б: Нереляційна база даних (NoSQL)**
 - Створіть діаграму або схему колекцій/документів.
 - Відобразіть структуру JSON-документів для основних сутностей.
 - Вкажіть стратегію зв'язків: **Embedding** (вкладення даних) чи **Referencing** (посилання на ID).
 - Обґрунтуйте вибір стратегії (наприклад, "Використано Embedding для коментарів, щоб отримати пост разом із коментарями за один запит").

Хід виконання

1. Обраний технологічний стек

Компонент	Технологія
Back-end	Node.js (Express)
Front-end	React
База даних	MongoDB
DevOps	GitHub

Для реалізації вебдодатку соціальної мережі з месенджером було обрано стек **JavaScript (MERN)**.

- **Node.js (Express)** використовується для бекенду, оскільки:
- добре підходить для роботи в реальному часі (чат)
- має велику кількість бібліотек
- дозволяє швидко створювати REST API

React обрано для фронтенду, тому що:

- дозволяє створювати SPA (Single Page Application)
- забезпечує швидкий та зручний UI
- добре підходить для динамічних інтерфейсів (чат, стрічка постів)

MongoDB використовується як база даних, оскільки:

- має гнучку структуру (зручно для постів і коментарів)
- добре працює з JSON-даними
- легко масштабується

2. Архітектура (C4 Model)

System Context Diagram (Контекст)



Рисунок 1 – Контекстна діаграма системи SocialHub

На рисунку представлено загальний контекст системи SocialHub. Система зображена як єдина сутність, що взаємодіє з основними акторами: гостем, користувачем та адміністратором. Також показано взаємодію із зовнішнім сервісом електронної пошти, який використовується для надсилання повідомлень та сповіщень.

Container Diagram (Контейнери)

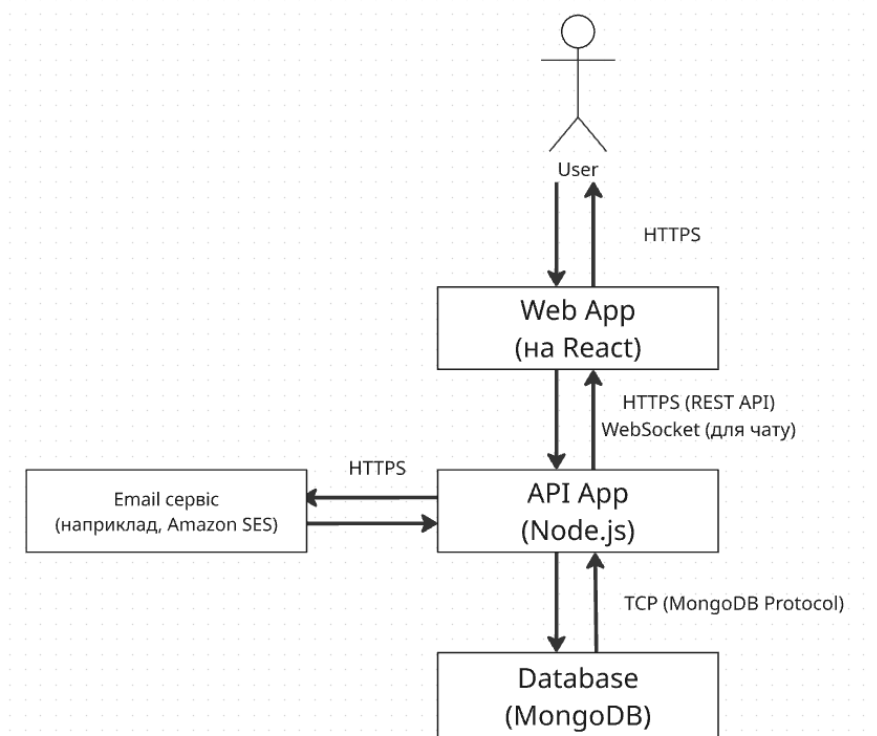


Рисунок 2 – Контейнерна діаграма системи SocialHub

Діаграма відображає внутрішню структуру системи SocialHub на рівні контейнерів. Система складається з клієнтського вебзастосунку (React), серверної частини (Node.js API) та бази даних MongoDB. Взаємодія між компонентами здійснюється через HTTPS для стандартних запитів та WebSocket для обміну повідомленнями в реальному часі. Backend також взаємодіє із зовнішнім email-сервісом через REST API. Взаємодія між backend та базою даних MongoDB здійснюється через TCP з використанням нативного протоколу MongoDB.

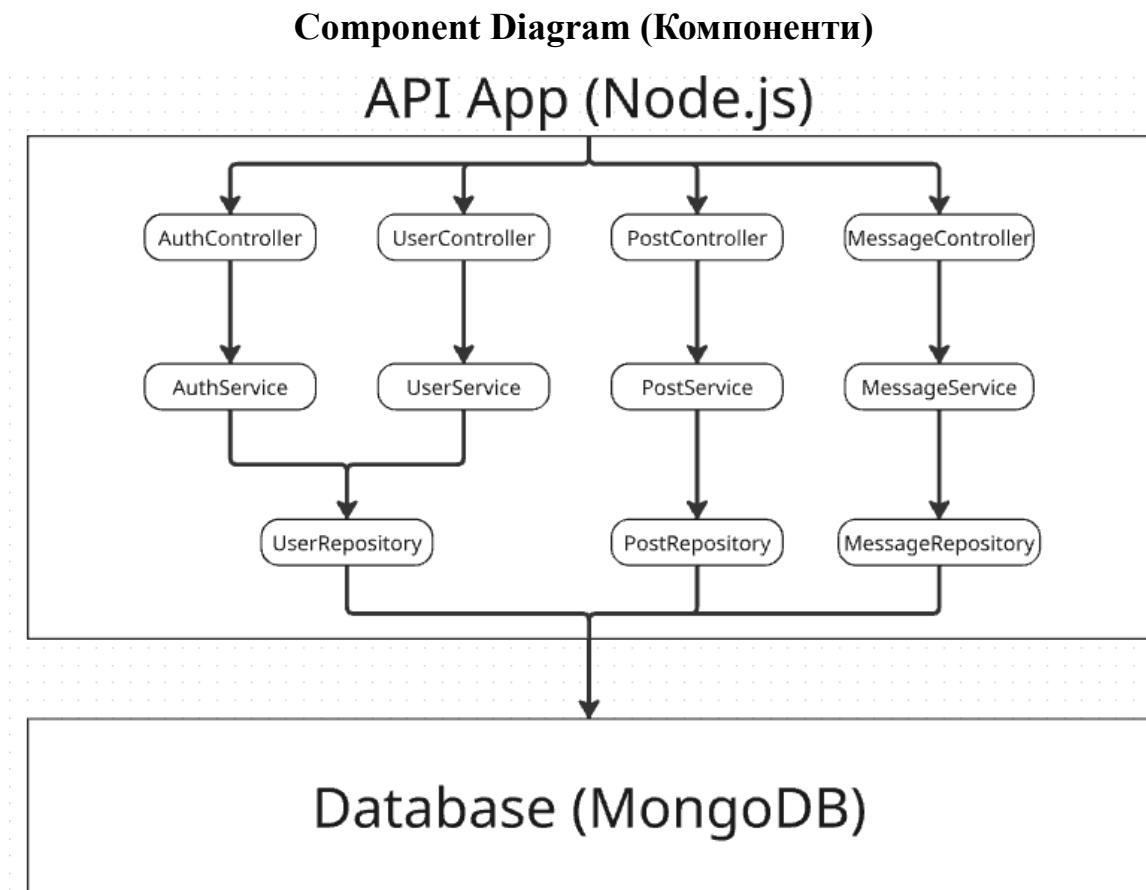


Рисунок 3 – Компонентна діаграма серверної частини системи SocialHub

На діаграмі представлено внутрішню структуру серверного застосунку. Архітектура побудована за принципом розділення відповідальностей і включає контролери, сервіси та репозиторії. Контролери обробляють HTTP-запити, сервіси реалізують бізнес-логіку, а репозиторії відповідають за взаємодію з базою даних MongoDB.

3. Проектування бази даних (MongoDB)

Обраний тип: NoSQL (MongoDB)

Колекції

Users

```
{
  "_id": ObjectId,
  "username": "string",
  "email": "string",
  "password": "string",
  "avatar": "string",
  "followers": [ObjectId],
  "following": [ObjectId]
}
```

Posts

```
{
  "_id": ObjectId,
  "userId": ObjectId,
  "content": "string",
  "createdAt": "Date",
  "likes": [ObjectId],
  "comments": [
    {
      "userId": ObjectId,
      "text": "string",
      "createdAt": "Date"
    }
  ]
}
```

Messages

```
{  
  "_id": ObjectId,  
  "senderId": ObjectId,  
  "receiverId": ObjectId,  
  "text": "string",  
  "createdAt": "Date"  
}
```

Стратегія зв'язків

- **Referencing (посилання)**

Використовується для:

1. `userId` у постах
2. `senderId`, `receiverId` у повідомленнях
3. списків `followers` і `following`

Дані зберігаються як посилання (`ObjectId`) на інші документи.

- **Embedding (вкладення):**

Використовується для:

1. коментарів у постах

Коментарі зберігаються всередині документа поста.

Обґрунтування вибору

Використання **Embedding** для коментарів дозволяє отримувати пост разом із усіма коментарями за один запит, що підвищує продуктивність і зменшує кількість звернень до бази даних.

Використання **Referencing** для користувачів і повідомлень дозволяє уникнути дублювання даних, забезпечує гнучкість структури та спрощує оновлення інформації про користувачів.

Такий комбінований підхід є оптимальним для соціальної мережі, оскільки поєднує продуктивність та масштабованість.

Контрольні запитання

1. Що таке модель C4 і для чого потрібен кожен з її рівнів (Context, Container, Component)?
2. У чому принципова різниця між реляційними (SQL) та нереляційними (NoSQL) базами даних? Коли варто обирати кожен з них?
3. Що таке теорема CAP (Consistency, Availability, Partition tolerance) і як вона впливає на вибір бази даних?
4. Які переваги та недоліки мікросервісної архітектури порівняно з монолітною (на рівні Container diagram)?

Відповіді на контрольні запитання

1. Модель C4 – це спосіб опису архітектури системи на різних рівнях деталізації, щоб усім було зрозуміло, як працює система.

Context (Контекст) – показує систему як «чорну скриньку» і її взаємодію з користувачами та зовнішніми системами. **Приклад:** користувачі, адміністратори, email-сервіси.

Container (Контейнери) – розкриває систему на великі блоки: вебзастосунок, API, база даних. Показує технології та протоколи. **Приклад:** React-клієнт, Node.js API, MongoDB, HTTPS/WebSocket.

Component (Компоненти) – деталізує контейнер на внутрішні частини: контролери, сервіси, репозиторії, модулі авторизації. **Приклад:** AuthController, PostService, UserRepository.

Коротко: Context – «що і з ким», Container – «з чого складається», Component – «як працює всередині».

2. SQL (реляційні) – дані зберігаються у таблицях, жорстка схема, складні зв'язки між таблицями, транзакції ACID. **Варто обирати для:** фінансових систем, CRM, складних бізнес-процесів, де потрібна строгість даних.

NoSQL (нереляційні) – дані у документах, ключ-значення або графах, гнучка схема, масштабування горизонтально. **Варто обирати для:** соціальних мереж, чати, каталогів з товарами, Big Data, коли структура часто змінюється і важлива швидкість.

3. Теорема CAP стверджує, що у розподіленій системі одночасно можна гарантувати тільки два з трьох параметрів: узгодженість даних (Consistency), доступність (Availability) та стійкість до розділення мережі (Partition tolerance). Це впливає на вибір бази даних: для соціальної мережі зазвичай жертвують частковою узгодженістю, щоб забезпечити доступність і роботу при розподілі, а для банківської системи обирають узгодженість і стійкість, щоб гарантувати точність транзакцій.

4. Мікросервісна архітектура дозволяє незалежно масштабувати та оновлювати сервіси, використовувати різні технології для кожного компоненту і підвищує відмовостійкість, але вимагає складної інтеграції, активного обміну повідомленнями між сервісами та розвиненого DevOps. Монолітна архітектура простіша у запуску та тестуванні, але складніше масштабується і оновлюється, оскільки всі функції об'єднані в одному контейнері.